

AI/ML-Based Adaptive Noise Cancellation for Defence Communications

Code Explanation & Frameworks Used

This is a companion reference to the Technical Approach & Jury Q&A document. Where that document argues the case, this one opens the project folder itself: what every script actually does, in the order it's actually used, and every third-party framework or library the project depends on and exactly why.

1. Frameworks & Libraries Used

Everything below is pinned in requirements.txt (torch, numpy, scipy, soundfile, onnx, onnxruntime, sounddevice) except the last row, which is an operating-system service definition rather than a Python package.

Framework / Library	What it actually does here
PyTorch	Defines and trains the neural network (model.py) and runs the laptop-side real-time rehearsal (live_test.py, enhance_test.py). Only needed on the training machine — never installed on the Raspberry Pi.
NumPy	All array math throughout the project: framing raw audio into overlapping windows, FFT-based frequency conversion, mask arithmetic, and the resampling math on the Pi. Used in essentially every file.
SciPy (scipy.signal.resample_poly)	High-quality polyphase resampling used once, specifically to fix speech/noise files that weren't recorded at 16kHz mono before training (fix_sample_rates.py).
SoundFile	Reads and writes WAV/FLAC files on disk — loading training clips, saving recorded noise samples, and writing the before/after test files.
sounddevice (PortAudio bindings)	Opens real-time microphone input and speaker output streams. Drives the laptop rehearsal, the actual Raspberry Pi runtime, and the manual noise-recording helper.
ONNX	The open, framework-independent file format the trained PyTorch model is converted into, so it can be run without PyTorch installed (export_onnx.py).
ONNX Runtime	The lightweight engine that actually executes the exported model on the Raspberry Pi — far smaller and faster to install than full PyTorch on constrained embedded hardware.
Python standard library (threading, wave, csv, glob, os, time, datetime, math, shutil)	Glue code: a background thread that listens for Enter-key presses to start/stop recording a take, writing 16-bit PCM WAV files by hand, per-epoch CSV training logs, discovering audio files on disk, and backing up

	originals before resampling.
systemd (sih-anc.service)	Not a Python library — an OS-level service unit that starts the Pi's real-time denoiser automatically on boot and restarts it automatically if it ever crashes, so the demo doesn't depend on someone SSH-ing in and running a script by hand.

2. The Two-Phase Structure the Code Follows

Every file in the project belongs to one of two phases, and the file-by-file walkthrough in section 3 is ordered the same way:

- **Phase A — Training** — done once, on a laptop/desktop with PyTorch installed. Reads speech and noise clips, mixes them, and repeatedly shows the network examples until it learns to predict the correct suppression mask. Produces one artifact that matters going forward: `denoise_net_best.pt`.
- **Phase B — Deployment** — runs continuously on the Raspberry Pi, using the already-trained, frozen model exported to ONNX. No learning happens here — it only ever converts incoming microphone audio into cleaned outgoing audio, block by block, in real time.

3. File-by-File Code Walkthrough

Listed in the order the files are actually used, not alphabetically.

3.1 Shared Building Blocks

data.py — constants + audio-to-training-example pipeline

Defines the four numbers every other file in the project has to agree on: 16kHz sample rate, a 512-sample FFT window, a 256-sample hop (16 ms — how often a fresh decision is made), and an 8-frame context window. If any of these drifted between files, a model trained with one setting would be fed data shaped for a different one.

```
data.py
SR = 16000
N_FFT = 512
HOP = 256
CONTEXT = 8    # how many recent frames of context the model gets to see
```

`load_mono()` reads a file and asserts it's already 16kHz (this is what `check_data.py` and `fix_sample_rates.py` exist to guarantee ahead of time). `mix_at_snr()` blends a speech clip with a random segment of a noise clip at a chosen loudness ratio. `spectrogram()` implements the STFT: it chops audio into overlapping 512-sample Hann-windowed frames, hopping forward 256 samples each time, and keeps only the magnitude of each frame's FFT — the "how much of each pitch is present" view the model actually looks at. `make_training_example()` ties it together: mix, convert both the mixture and the original clean speech to this frequency form, and compute the mathematically correct answer (the ideal ratio mask) since the clean speech and noise are still known separately before mixing.

```
data.py — the ideal ratio mask formula
ideal_mask = clip( clean_mag / (clean_mag + noise_mag + 1e-8), 0.0, 1.0 )
```

model.py — the network's structure

```
model.py (abridged)
class DenoiseNet(nn.Module):
    def __init__(self, n_freq=257, hidden=64):
        self.gru = nn.GRU(input_size=n_freq, hidden_size=hidden, batch_first=True)
        self.fc = nn.Linear(hidden, n_freq)

    def forward(self, x):
        # x: (batch, CONTEXT, n_freq)
        out, _ = self.gru(x)
        mask = torch.sigmoid(self.fc(out[:, -1, :])) # (batch, n_freq) in (0,1)
```

A single GRU (Gated Recurrent Unit) layer with 64 hidden units, followed by one linear layer and a sigmoid, so every one of the 257 output values is squeezed into $[0, 1]$ — the suppression mask for the newest frame. A GRU is a recurrent network with a short built-in memory of what it just processed, which matters because separating voice from noise often depends on how a sound changes over the last few frames, not just the current instant.

Working through `nn.GRU`'s parameter shapes for `input_size=257`, `hidden_size=64` gives $3 \times 64 \times (257 + 64 + 2) = 62,016$ parameters in the GRU, plus $64 \times 257 + 257 = 16,705$ in the output layer — about 78,700 trainable parameters in total. That's tens of thousands, not millions: small enough to run in real time on a Raspberry Pi CPU with no GPU or accelerator, in the same size class as `RNNNoise`'s core network.

3.2 Training Pipeline

check_data.py & fix_sample_rates.py — data housekeeping

Not part of the AI itself. `check_data.py` scans `data/speech` and `data/noise`, reports how many files are there, and flags any that aren't 16kHz mono before time gets wasted training on broken data. `fix_sample_rates.py` repairs what it finds — using `scipy.signal.resample_poly` for proper polyphase resampling rather than a naive method — and always backs up the original file into a sibling `_originals` folder first, so a bad conversion is never a one-way trip.

record_noise.py — capturing custom noise clips

A small utility to record a new noise sound straight from the microphone into `data/noise/`, used to add sounds the public datasets didn't cover (finger snaps, keyboard typing, and similar) so the very next training run can include them.

train.py — the training loop

Loads every speech/noise file into memory once, splits files (not individual examples) 85% / 15% into training and validation sets so the model is checked against speakers and noises it never trained on, then repeats the process from `data.py` thousands of times per epoch: generate a fresh random mixture, ask the model to guess the mask, compare it to the known correct answer with mean-squared-error loss, and nudge the model's weights a little closer via the Adam optimizer. After every epoch it saves the model's weights to `denoise_net_best.pt`, but only if that epoch actually improved on the best validation score seen so far — so the file you actually use is never accidentally a worse, later epoch.

```
train.py - current config
FINE_TUNE_FROM_EXISTING = True # continue an existing checkpoint, don't start over
EPOCHS = 12                  # a top-up run needs far fewer epochs than scratch
BATCH_SIZE = 32
SNR_RANGE_DB = (-5, 15)      # random noisy-vs-clean loudness ratio
```

One detail worth knowing if it comes up: the script's current defaults are tuned for topping up an already-trained model with new noise types quickly (12 epochs, a gentler fine-tuning learning rate) rather than

training from nothing. If `denoise_net_best.pt` doesn't already exist, it automatically falls back to a full from-scratch run instead — which is the mode used for the original 35-epoch training run referenced elsewhere in the project's numbers, before this fine-tuning shortcut existed.

3.3 Validation & Testing (Laptop)

enhance_test.py — offline before/after test

Takes one speech clip and one noise clip, mixes them, runs the already-trained model over the whole mixture at once (with the luxury of processing frame-by-frame but never in a live stream), and reconstructs the enhanced waveform with the inverse STFT via overlap-add. Writes `test_noisy.wav` and `test_enhanced.wav` so the two can be compared by ear. This is the first sanity check run after every training run, before trusting the model live.

live_test.py — the real-time microphone rehearsal

The important one to understand deeply, because it's a laptop dress rehearsal of exactly what runs on the Raspberry Pi. Unlike `enhance_test.py`, this never sees a whole file at once — it processes one 16 ms block at a time, using only audio it has already heard. The `StreamingDenoiser` class keeps three small rolling buffers between calls: the tail of raw samples needed to build a full analysis frame from successive small blocks, a rolling context of recent magnitude frames (what the model actually looks at), and an overlap-add accumulator for reconstructing smooth output audio a block at a time instead of a whole file at once.

Two tuning controls were added here after listening to real recordings, and they apply after the model's raw output without needing any retraining:

```
live_test.py
MASK_GAMMA = 2.0    # sharpens the mask: raises low (noisy) values further toward 0
MASK_FLOOR = 0.08   # any mask value below this is forced to complete silence

def sharpen(mask, gamma=MASK_GAMMA, floor=MASK_FLOOR):
    m = clip(mask, 0.0, 1.0) ** gamma
    return where(m < floor, 0.0, m)
```

A third control, automatic gain compensation (AGC), exists because the model can only ever remove energy, never add it — so correctly-working suppression still leaves the output quieter than the input. AGC tracks a short-term running loudness of the input versus the (pre-gain) output and boosts the output back up to compensate, capped by `AGC_MAX_GAIN` so near-silence never gets amplified into audible hiss.

3.4 Deployment Preparation

export_onnx.py — preparing the model for the Pi

Loads the trained `denoise_net_best.pt` weights, runs a single dummy input through `torch.onnx.export` to convert the model into the portable ONNX format, then immediately sanity-checks the conversion itself: it runs the same random input through both the original PyTorch model and the freshly exported ONNX model and prints the maximum difference between the two outputs. In practice this comes back at roughly 1.64×10^{-7} — effectively bit-identical — which is the evidence that exporting to ONNX introduces no accuracy loss before ever touching the Pi. The export deliberately forces the legacy TorchScript-based exporter (`dynamo=False`) rather than newer PyTorch's default exporter, since this simple GRU + Linear model has no dynamic control flow and the legacy path avoids an extra dependency (`onnxscript`) that isn't otherwise needed.

3.5 Raspberry Pi Runtime

pi_model_smoke_test.py — no-audio model check

Run on the Pi immediately after copying `denoise_net.onnx` over, before any microphone or speaker is even connected. Loads the model through ONNX Runtime and fires 20 inference passes with random input to confirm the output shape and value range are correct, and to measure timing directly on the Pi's own CPU. This is the source of the headline performance numbers used elsewhere — 0.27 ms average / 1.52 ms maximum per 16 ms audio frame — and the script explicitly compares that against the real-time budget ($256/16000 \times 1000 = 16.0$ ms per frame) and prints a warning if inference were ever too slow to keep up.

pi_output_test.py — isolating the audio hardware

A minimal test that plays a plain 440Hz tone through the Pi's 3.5mm jack — no model, no microphone — to confirm `sounddevice/PortAudio` can actually drive that output during live playback, isolated from every other moving part. Written specifically to separate "is the AI pipeline broken" from "is the audio hardware/driver path broken," which turned out to matter (see `pi_live_test.py` below).

pi_live_test.py — the real-time denoiser that actually runs on the Pi

The ONNX Runtime equivalent of `live_test.py` — same causal, block-by-block algorithm (`StreamingDenoiserONNX` mirrors `StreamingDenoiser` almost line for line, with an `onnxruntime.InferenceSession.run()` call in place of the PyTorch forward pass), same `MASK_GAMMA / MASK_FLOOR / AGC` tuning, same manual take-recording via Enter key. This is the script `sih-anc.service` actually launches.

The one genuinely tricky piece of engineering in the whole project lives here. The USB headset mic and the Pi's own 3.5mm jack are two completely separate physical sound cards with no shared hardware clock. Running them as a single combined input+output stream (the obvious first approach) turned out to be unreliable on `ALSA/PortAudio` — it could silently produce corrupted or completely inaudible live audio with no error reported, even though the exact same processed audio was correct when saved to a WAV file and the output hardware worked fine driven on its own. The fix was to stop trying to force one shared clock onto two independent devices: run two fully independent streams (an `InputStream` on the mic, an `OutputStream` on the jack), each on its own callback thread, connected through a small thread-safe ring buffer (`AudioRelay`) that the input side pushes into and the output side pulls from whenever its hardware wants more samples — with a hard cap on how much audio the buffer is allowed to hold, so if the two clocks drift, the fix is dropping the oldest samples to keep latency bounded, never letting it grow unboundedly.

A second practical detail: the USB mic and the Pi's jack each report their own native hardware sample rate (often 44100 or 48000 Hz, not the model's fixed 16000 Hz), so every block is resampled in both directions with a small dependency-free linear-interpolation resampler (`resample_block`) — not as pristine as `scipy's` polyphase resampler used offline for training data, but adequate at these block sizes and avoids needing `scipy` on the Pi runtime path at all.

compare_takes.py — proving the effect numerically

Loads a saved input/enhanced WAV pair and reports whether they're bit-for-bit identical, the maximum sample difference, and the RMS (loudness) of each — specifically so a short or quiet noise burst that's hard to judge by ear can still be confirmed as "the pipeline is or isn't actually doing something" with a number, not a guess. This is also what produced the peak-amplitude and RMS-energy reduction percentages quoted for real gunshot transient tests elsewhere in the project.

sih-anc.service — running it unattended

```
sih-anc.service (abridged)
[Service]
ExecStart=/home/<user>/anc-venv/bin/python /home/<user>/sih-anc/pi_live_test.py
Restart=on-failure
RestartSec=3

[Install]
```

```
WantedBy=multi-user.target
```

A systemd unit file, not Python code — it tells the Pi's operating system to start `pi_live_test.py` automatically on boot inside the project's virtual environment, and to restart it automatically after any crash. This is what makes the demo resilient to a power cycle or an unhandled error without anyone needing to SSH in and re-run a command by hand.

4. Key Algorithms & Design Choices, in Plain Language

Short-Time Fourier Transform (STFT)

The standard, decades-old technique for turning a raw sound wave into a "how much of each pitch is present, moment by moment" view. Nothing experimental about it — it's just chopping audio into small overlapping windows and running an FFT on each one. The model never looks at the raw waveform, only this frequency view, because voice and noise tend to occupy different, more separable pitches.

Ideal Ratio Mask

The model's entire output is 257 numbers between 0 and 1 — one per frequency bin — representing how much of that bin to keep. It never generates or invents audio; it only ever decides what fraction of what's actually there to keep versus discard. This is the same general family of technique used by RNNoise and DeepFilterNet, the two published systems this project's approach is closest to.

Causal, block-by-block processing

Every real-time script processes audio using only what has already happened, with no ability to look ahead — this is what makes real-time noise cancellation fundamentally harder than cleaning up a pre-recorded file, and it's why very sudden, sharp sounds always leak through slightly at the exact instant they begin, in this system and in production tools like RNNoise alike.

Overlap-add reconstruction

Once the mask is applied to a frame's spectrum, converting back to a waveform with an inverse FFT produces one short windowed chunk at a time. Overlap-add stitches these chunks back into one continuous signal by summing overlapping windowed frames and dividing by the accumulated window energy (window-sum normalization) — the standard, artifact-free way to reverse framing, validated by confirming that an all-ones mask reconstructs the original input to numerical precision.

Why ONNX Runtime instead of PyTorch on the Pi

PyTorch is built for training — flexible, large, and full of functionality (autograd, optimizers, dataset loaders) the Pi will never use once the model is frozen. ONNX Runtime is a much smaller, inference-only engine that runs the exact same trained weights. This is standard practice for deploying a trained model to constrained embedded hardware, not a shortcut specific to this project.

5. Exact Parameters Reference

Setting	Value (from the code)
Sample rate (SR)	16,000 Hz — <code>data.py</code> , shared by every file
FFT window size (<code>N_FFT</code>)	512 samples (32 ms) — <code>data.py</code>

Hop size (HOP)	256 samples / 16 ms — data.py; also the real-time frame rate
Frequency bins (N_FREQ)	$257 = N_FFT // 2 + 1$ — data.py, model.py
Context window (CONTEXT)	8 frames (~144 ms of history) — data.py
Model architecture	GRU($257 \rightarrow 64$) + Linear($64 \rightarrow 257$) + sigmoid — model.py, ~78,700 parameters
Batch size	32 — train.py
Training epochs (current script default)	12, tuned for fine-tuning an existing checkpoint — train.py
Train/validation split	85% / 15%, split by file not by example — train.py
SNR range used in training	-5 dB to +15 dB, randomized per example — train.py
Mask sharpening (MASK_GAMMA)	2.0 — live_test.py, pi_live_test.py
Mask floor (MASK_FLOOR)	0.08 — live_test.py, pi_live_test.py
ONNX opset	17 — export_onnx.py
PyTorch vs. ONNX output difference	$\sim 1.64 \times 10^{-7}$ (effectively bit-identical) — export_onnx.py's built-in check
Measured Pi inference time	0.27 ms avg / 1.52 ms max per 16 ms frame — pi_model_smoke_test.py

6. How to Use This Document

Every number and code excerpt above was taken directly from the scripts in the project folder, not from memory or from the narrative planning notes — if the code changes, this document should be regenerated from it rather than hand-edited to match. Pair this with the Technical Approach & Jury Q&A document for the case for why these choices were made; this one is for exactly what was built.